Architecture Justification – Campus Placement & Internship Pipeline

Lab 3: Component Modelling & Architectural Pattern Selection

NAME: Chennamraju Vaishnavi

SRN: PES1UG24CS129

SECTION: C

Architecture Selection: We chose Layered Architecture for the Campus Placement & Internship Pipeline System.

1. Architectural Choice

The system is structured into three layers: Presentation (User Interface Component used by Student Applicants and Placement Officers), Business (Resume Parser, Eligibility Filter, Interview Queue Manager, Offer Letter Processor), and Data (Student Academic Records DB and Drive & Offer Records DB). This layered style cleanly separates resume parsing, multi-tier CGPA/backlog filtering, interview-queue management, and offer-letter processing while keeping academic records and offer documents protected in the data layer.

2. Two Scenario-Related Reasons for Selection

Reason 1 – Clear separation of concerns for complex eligibility rules: Campus drives impose multi-tier CGPA and backlog filters that change per company. Layered architecture isolates the Eligibility Filter Component so that rule changes affect only the business layer; the UI and database remain stable. Resume parsing, interview-queue logic, and offer generation are likewise independent modules, making the system easier to maintain when hundreds of corporate drives run concurrently.

Reason 2 – Centralised data consistency and simpler deployment: Student academic records and offer letters must remain consistent and auditable. A layered design keeps a single logical data store, avoiding the distributed-data consistency problems of microservices. Deployment stays straightforward for a campus placement cell that typically runs one primary platform rather than many independently scaled services.

3. Security Advantage

Student academic records and offer documents are confined to the Data layer and accessed only through narrow interfaces (IStudentData, IOfferStore). The Presentation layer never reads raw academic tables; it receives only filtered results via the business components. This supports the non-functional requirement to encrypt academic records and offer documents at rest (AES-256): encryption can be enforced inside the data-layer components while business and presentation layers remain unaware of storage details, reducing the attack surface if the UI is compromised.

4. Performance Benefit

Eligibility checks and resume parsing occur as in-process calls within the business layer, avoiding network latency that microservices would introduce. When hundreds of students apply to a drive, the Eligibility Filter can batch CGPA/backlog queries against the Student Academic Records DB efficiently. The layered design still permits future caching of eligibility results or parallel processing of resume parsing without redesigning the upper or lower layers.

5. Components & Interfaces Realised

Seven components: User Interface, Resume Parser, Eligibility Filter, Interview Queue Manager, Offer Letter Processor, Student Academic Records DB, Drive & Offer Records DB. Key interfaces: IPortalAccess, IApplication / IResumeParse, IEligibility, IInterview, IOffer, IStudentData, IDriveData, IOfferStore (provided/required ball-and-socket notation).